

Mongodb

***MongoDB** is an open source NOSQL dataset in C++, this offers flexible data models, indexing, replication and modern applications for scalable architecture.*

NoSQL stands for "**Not Only SQL**."

Unlike relational databases, NoSQL stores data in flexible formats such as:

- Documents
- Key-Value
- Graph
- Column-based

MongoDB is a **Document-Oriented NoSQL Database**.

Features

- Flexible Schema
- High Performance
- Easy to Scale
- JSON-like Documents
- Cloud Friendly

SQL	NoSQL
Tables	Collections
Rows	Documents
Columns	Fields
Fixed Schema	Flexible Schema
MySQL	MongoDB

Advantages of MongoDB

- ✓ Easy to Learn
- ✓ Stores JSON-like Documents
- ✓ High Performance
- ✓ Horizontal Scaling
- ✓ Fast Query Execution
- ✓ Flexible Structure
- ✓ Widely used with MERN Stack

Companies Using MongoDB

- Adobe
- Uber
- eBay
- Coinbase
- Bosch

Commands

Show All Databases

```
show dbs
```

Create or Switch Database

```
use collegeDB
```

Check Current Database

```
Db
```

Create Collection

```
db.createCollection("students")
```

Show All Collections

```
show collections
```

Insert One Document

```
db.students.insertOne({  
  name: "Rahul",  
  age: 22,  
  course: "React JS"  
})
```

Insert Multiple Documents

```
db.students.insertMany([  
  {  
    name: "Amit",  
    age: 21,  
    course: "Node JS"  
  },  
  {  
    name: "Priya",  
    age: 23,
```

```
        course: "MERN Stack"
    }
  })
```

Find All Documents

```
db.students.find()
```

Find One Document

```
db.students.findOne()
```

Find with Condition

```
db.students.find({
  age: 22
})
```

Find Specific Fields

```
db.students.find({}, {
  name: 1,
  age: 1,
  _id: 0
})
```

Comparison Operators

Equal to {<key>:{\$eq:<value>}}

Less Than {<key>:{\$lt:<value>}}

Less than equal {<key>:{\$lte:<value>}}

Greater Than : {<key>:{\$gt:<value>}}

Greater Than or equal {<key>:{\$gte:<value>}}

Not Equal {<key>:{\$ne:<value>}}

Values in an array {<key>:{\$in:[<value1>, <value2>,<valueN>]}}

Values not in an array {<key>:{\$nin:<value>}}

AND in MongoDB

```
db.mycol.find({ $and: [ {<key1>:<value1>}, { <key2>:<value2>} ] })
```

OR in MongoDB

```
db.mycol.find({$or: [{key1: value1}, {key2:value2}]}).pretty()
```

NOT in MongoDB

```
db.COLLECTION_NAME.find(  
    {$not: [{key1: value1}, {key2:value2}] }  
)
```

Update One Document

```
db.COLLECTION_NAME.update(SELECTION_CRITERIA, UPDATED_DATA)
```

MongoDB findOneAndUpdate() method

```
db.COLLECTION_NAME.findOneAndUpdate(SELECTION_CRITERIA, UPDATED_DATA)
```

updateOne()

```
db.COLLECTION_NAME.updateOne(<filter>, <update>)
```

updateMany()

```
db.COLLECTION_NAME.update(<filter>, <update>)
```

deleteOne()

```
db.COLLECTION_NAME.deleteOne(DELETION_CRITERIA)
```

Remove All Documents

```
db.mycol.remove({})
```

Count Documents

```
db.students.countDocuments()
```

Sort Documents

```
db.students.find().sort({ age: 1 })           :      ASC
```

```
db.students.find().sort({ age: -1 })          :      DESC
```

Drop Collection

```
db.students.drop()
```

Drop Database

```
db.dropDatabase()
```

Limit Records

```
db.students.find().limit(3)
```

Skip Records

```
db.students.find().skip(2)
```

Pagination

```
db.students.find().skip(5).limit(5)
```